

Operációs rendszerek

Folyamatok közötti kommunikáció (Inter Process Communication)

Processzusok közötti kommunikációval kapcsolatosan felmerülő kérdések, elvárások:

- Hogyan tud egy processzus információt cserélni egy másik processzussal (adatcsere)
- Két vagy több processzus ne tudja egymás útját keresztezni
- Processzusok sorrendbe állítása ha az **A processzus** állítja (készíti) elő az adatokat és a **B processzus** kell feldolgozza akkor a B processzusnak várni kell az A-ra

Kommunikációs szálak esetében

- a **közös címtartománnyal** rendelkező szálak esetében egyszerűen megoldható az adatcsere
- **Különböző címtartománnyal rendelkező szálak** esetében a szálak közötti kommunikációra alkalmazhatóak a processzusok közötti kommunikáció során alkalmazandó megoldások

- A processzusok kommunikációja során **versenyhelyzet** alakulhat ki a processzusok között.
- Ha két vagy több processzus versenyez, ugyanazért az erőforrásért azt mondjuk, hogy **versenyhelyzet alakult** ki.

Folyamatok közötti kommunikáció

- **kritikus szekció**
- **kölcsönös kizárás**
- A versenyhelyzetek megelőzésére szükségünk van a **kölcsönös kizárásra**, azaz arra, hogy más processzusokat távol tartsunk a közös erőforrásoktól, amíg azokat egy **futó processzus** használja.

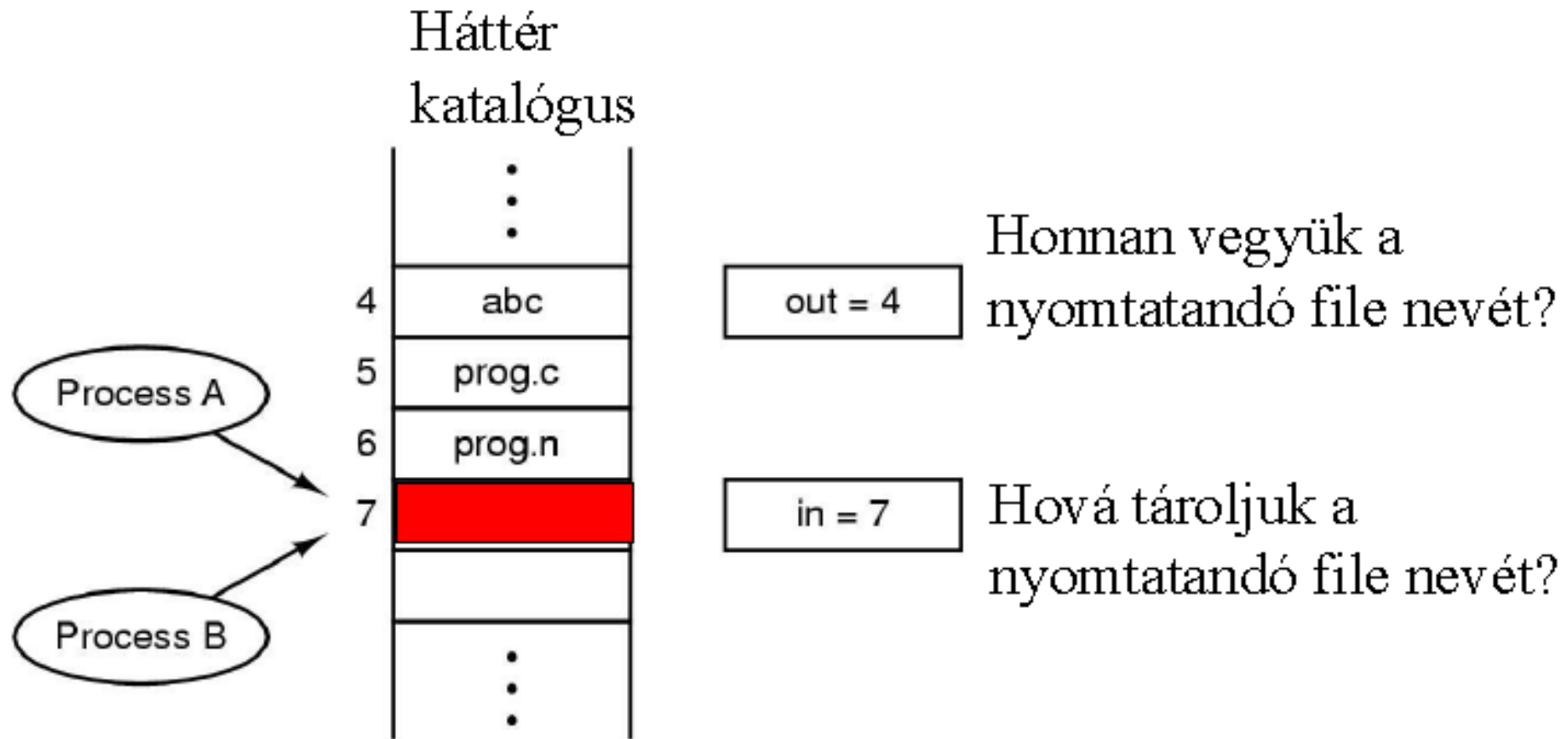
A programnak azt a részét, amelyben **megosztott erőforrást** használ, **kritikus területnek**, illetve **kritikus szekciónak** nevezzük.

- A **kölcsönös kizárás**: mialatt egy processzus a saját **kritikus szekciójában** a **megosztott erőforrást** aktualizálja, ne legyen más olyan processzus, amely saját kritikus szekciójába lép, **ugyanazt az erőforrást használná**, (a rendszer hibás működését okozhatja).
- Az operációs rendszerek tervezésének fontos szempontja, hogy jól megyszerkesztett **primitív műveleteket** alkalmazzanak a kölcsönös kizárás megvalósításához.

Milyen megoldás fogadható el?

- **ne legyen egyszerre két processzus a saját kritikus szekciójában,** amelyek egy közös erőforrást osztanak meg
- semmilyen előfeltétel ne legyen a **sebességekről** vagy a CPU-k számáról
- egyetlen **kritikus szekción kívül levő processzus** sem blokkolhat más processzusokat
- egyetlen processzusnak sem kelljen **örökre arra várni, hogy belépjen a kritikus szekcióba**

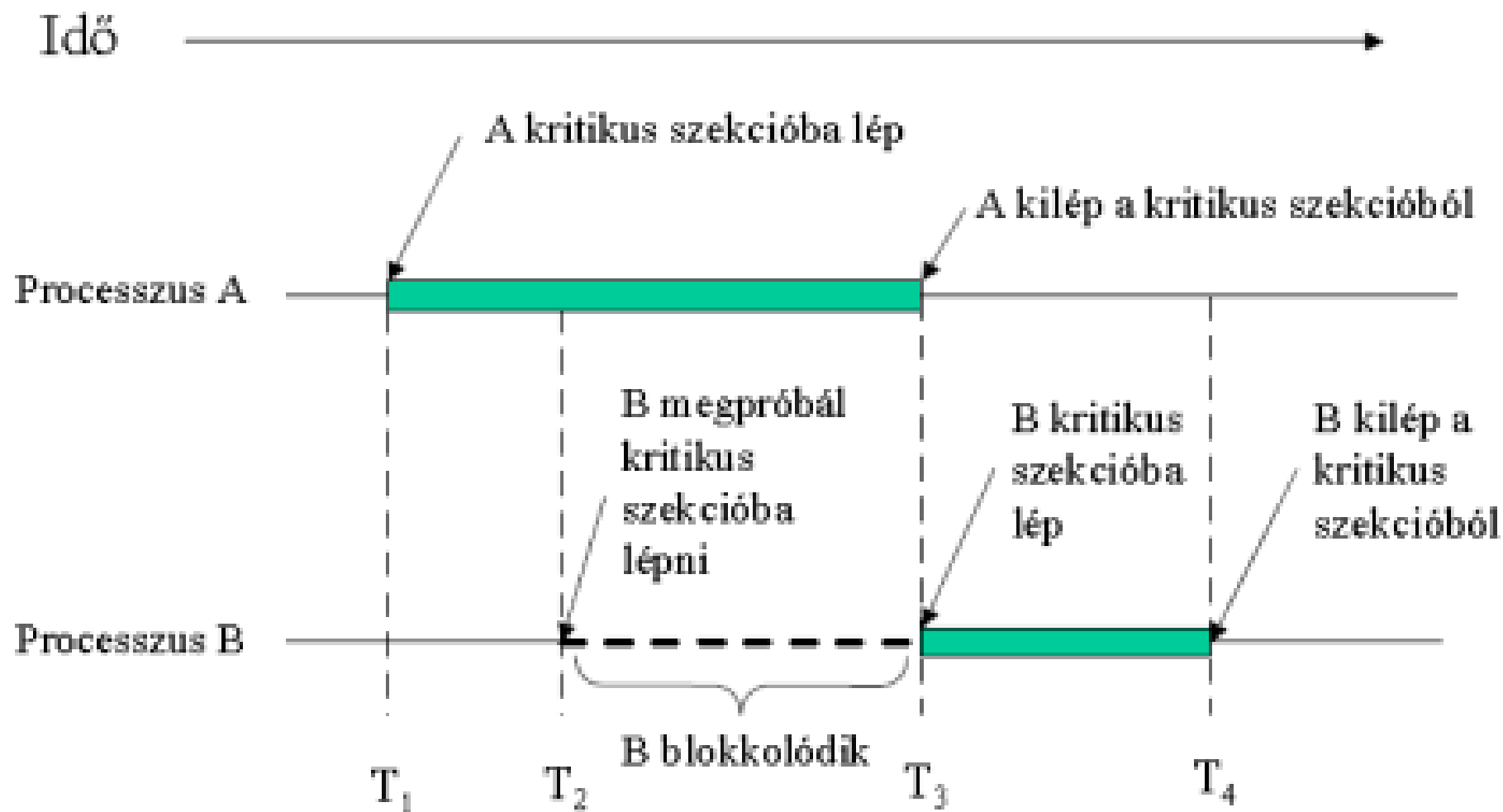
Mi történik, ha egy processzus kiszerezetne nyomtatni egy fájlt?



Melyik rész lenne ebben az esetben a kritikus rész?

- ha egyik processzus elkezd írni a háttértárolóba, addig a másik (vagy többi processzus) ne írhatson amíg az első be nem fejezte teljesen az írást
 - kiolvassa a következő szabad rekesz számát (indexét)
 - a szabad rekesz mezejébe beírja a nyomtatandó fájlt
 - növeli a szabad rekesz indexét 1-el

Kritikus szekció



- T1 első processzus belép a kritikus részbe
- T2-a második processzus megpróbál belépni a kritikus részbe (nem sikerül és blokkolódik)
- T3-az első (A) processzus kilép a kritikus szekcióból és belép a B processzus
- T4-a B kilép a kritikus szekcióból

Folyamatok közötti kommunikáció

- jelzések
- állományok
- csővezetékek
- névvel ellátott csővezetékek
- semaforok
- üzenetsorokkal
- osztott memória
- socket,
- stre.

Zárolás változók

- rendelkezésre áll egy **zárolásváltozó (flag)**, egy megosztott változó, amely elérhető mindkét processzusból
- ha a változó értéke nulla, azt jelenti, hogy egyetlen processzus sincs a kritikus szekcióban
- ha a változó értéke 1 (vagy nullától különböző) egy processzus a kritikus részben van, ha a változó értéke nulla egyesre állítja a változót és belép a kritikus szekcióba
- belépéskor, ha már a változó értéke 1, a processzus addig vár amíg a változó értéke zérus nem lesz
- kilépéskor a zárolás változó értékét vissza állítja nullára, lehetővé téve, hogy egy másik belépésre váró folyamat belépjen a kritikus szekcióba

```
while (true)
{while(flag);
flag = 1;
kritikus_szekcio();
flag = 0;
nemkritikus_szekcio();
}
```

```
while (true)
{while(flag);
flag = 1;
kritikus_szekcio();
flag = 0;
nemkritikus_szekcio();
}
```

Szigorú váltogatás

```
while(TRUE)
{while(flag != 0);
kritikus_szekcio();
flag=1;
nemkritikus_szekcio();
}
```

```
while(TRUE)
{while(flag != 1);
kritikus_szekcio();
flag=0;
nemkritikus_szekcio();
}
```

Mi a hibája ennek a megoldásnak?

- a tevékeny várakozást nem ajánlatos használni, mert pazarolja a CPU idejét. (elfogadható abban az esetben, ha rövid ideig kell csak várakozni)

Megszakítás Tiltása

- a processzus letiltja az összes megszakítást amikor belép a kritikus szekcióba és engedélyezi a megszakításokat kilépéskor
- nem történik processzus váltás, a “kritikus részben” elvégezheti a feladatát anélkül, hogy egy másik processzus közbeszólna
- a megszakítások felhasználói kézbe kerülnek
- mi történik ha például egy processzus kikapcsolja a megszakításokat és soha nem kapcsolja vissza
- megszakítások tiltása hasznos az operációs rendszeren belül, de nem hasznos a felhasználói processzusok számára

```
#define FALSE 0
#define TRUE 1
#define M 2
int flag
int erdekelt_proc[M];
void belepes(int processzus)
{
    int mas;
    mas=1-processzus;
    erdekelt[processzus]=TRUE;
    flag=processzus;
    while((flag == processzus) && (erdekelt_proc[mas]) == TRUE)
}
```

Különböző esetek szemléltetése

- A processzus érdekelt
- B processzus nem érdekelt(de a B megpróbál belepni mikor mar az A belepett)

```
A processzus(0)
void belepes(0)
{
int mas;
mas = 1-0; //mas = 1-processzus
erdekelt [0]= TRUE
flag = 0;
while((flag== 0) && (erdekelt[1]== TRUE));
}
```

* flag == 0 feltétel igaz de mivel a B processzus nem érdekelt az erdekelt[1]==TRUE feltétel nem teljesül . Az összfeltétel nem teljesül és a while ciklusból kilép a processzus. Az A processzus belép a kritikus szekcióba.

Mi történik a következő lépésben amikor megpróbál belépni a B processzus?

```
void belepes(1)
{
int mas;
mas = 1-1  //---
erdekelt[1]=TRUE;
flag=1;
while((flag == 1) && (erdekelt[0]= TRUE));
}
```

Ebben az esetben a feltétel igaz és mindaddig várakozni fog, amíg az A processzus nem hívja meg a kilepes() függvényt és az A processzusra az edekelt[0] változó értékét HAMIS-ra nem állítja edekelt[0]=FALSE

Mindkét processzus majdnem egyszerre próbál belépni a kritikus részbe

```
void belepes(0)
{
    int mas;
    mas = 1-0; //mas =1
    erdekelt[0]=TRUE;
    flag=0;
    while(flag == 0 && erdekelt[1] == TRUE)
    }
```

```
void belepes(1)
{
    int mas;
    mas = 1-1; //mas =0
    erdekelt[1]=TRUE;
    flag=1;
    while(flag == 1 && erdekelt[0] == TRUE)
    }
```

- Az A processzus esetében a (flag==0) nem teljesül (feltételezve, hogy a B processzus próbált másodjára belépni kritikus részbe a while ciklus teszt csak egyszer hajtódik végre)
- A B processzus várakozni fog, mivel mindkét feltétel teljesül.
- Amikor az A processzus kilép a kritikus szekcióból és végrehajtja (lefuttatja kilepes() függvényt és FALSE-ra állítja az érdekelt[0] változót. A B processzusban a while ciklus feltétele a következő ellenőrzéskor nem teljesül és a B processzus is beléphet a kritikus szekcióba)
- Ennek a megoldásnak is a hátránya a tevékeny várakozás

TSL utasítás (Test and Set Lock)

- a zárolás változók esetében a gond feloldódna, ha egyetlen utasítás alatt le lehetne ellenőrizni egy változó értékét és ugyanabban az utasításban ki is cserélni az értéket
- erre szolgálhat a TSL(Test and Set Lock) utasítás
- beolvassa a LOCK memóriaszó tartalmát az RX regiszterbe és ezután egy nem nulla értéket ír arra a memóriacímre
- a szó kiolvasása és tárolása művelet nem választható szét, az utasítás befejezéséig más processzus nem érheti el a memóriaszót

TSL RX LOCK

Belepes: TSL RX,LOCK

//átmásolja a LOCK tartalmát a regiszterbe és a
// LOCK értéket 1 re állítja

CMP RX,0

//LOCK nulla volt-e?

JMZ belepes

//ha nem volt nulla, akkor már egy másik
//processzus átállította

RET

//visszatérés a hívóhoz, sikeres belépés a kritikus
//szekcióba

Kilepes:

MOVE LOCK, 0

//A LOCK változót átállítjuk nullára

RET

A megoldások hibája a tevékeny várakozás

- amikor egy processzus beakar lépni a kritikus szekcióba ciklikusan ellenőrzi, hogy a belépés engedélyezett-e?
- Ha nem engedélyezett ciklusban marad az engedélyre várva

Két primitív utasítás (rendszerhívás) a processzus közötti kommunikációra

Sleep()- a hívót blokkolja amíg egy másik processzus fel nem ébreszti

Wakeup(pid_t)- a wakeup paramétere amelyik processzust fel kell ébreszteni

TERMELŐ FOGYASZTÓ probléma

A két processzus **osztozik** egy **közös rögzített méretű tárolón**. A **gyártó adatokat helyez** el a tárolóban. A **fogyasztó kiolvassa** és feldolgozza a tárolóban található adatokat.

A feladat általánosítható ' m ' gyártóra és ' n ' fogyasztóra (de mi most csak egy gyártóra és egy fogyasztóra tárgyaljuk a feladat megoldását).

Mi történik a gyártó szempontjából, ha a tároló megtelt? Mi történik a fogyasztó szempontjából, ha a tároló kiürült? Ha a tároló megtelt a gyártó elalszik, a fogyasztó amikor kivesz egy elemet felébreszti a gyártót. Ha a tároló kiürült a fogyasztó elalszik, a gyártó mikor egy új elemet tesz a tárolóba felébreszti a fogyasztót.

TERMELŐ FOGYASZTÓ probléma

```
void termelo(void)
```

```
{
```

```
int elem;
```

```
while(TRUE)
```

```
{elem=elemet_gyart();
```

```
If (szamlalo== M) sleep();
```

```
elemet_beszur();
```

```
szamlalo=szamlalo+1;
```

```
if(szamlalo >= 1)
```

```
wakeup(fogyaszto);
```

```
}
```

```
}
```

```
void fogyaszto()
```

```
{
```

```
int elem;
```

```
while(TRUE)
```

```
{
```

```
if(szamlalo == 0) sleep();
```

```
elem=elemet_kivesz();
```

```
szamlalo=szamlalo-1;
```

```
if (szamlalo<=M-1)
```

```
wakeup(termelo);
```

```
elemet_elfogyaszt();
```

```
}
```

```
}
```

Elküldök egy ébresztőt de a processzus amelynek elküldtem még nem aludt, ebben az esetben az ébresztés el fog vesztődni.

MEGOLDÁS:

ébresztőt váró bit hozzáadása:

ha volt ébresztés de a processzus még nem aludt beállítódik az ébresztést váró bit.

Amikor a processzus elakar aludni ellenőrzi az ébresztő bitet , ha az be volt állítva törli és ébren marad.

Szemaforok

- Dijkstra (1965) szemafor: pozitív egész számot tartalmazó közös változó és egy várakozási sor
- Atomi művelet: Egy időben csak egy processzus operálhat egy szemaforon.

Szemaforon két *atomi* operáció:

szemafor lefoglalása $\text{Down}(P)$

szemafor elengedése $\text{Up}(P)$

$\text{Down}(P)$

- a folyamat a **kritikus szekcióba való belépéskor** meghívja a **$\text{Down}(P)$** rendszerhívást
- ha foglalt a szemafor blokkolódik a folyamat
- A blokkolódásból “felébredő” processzus végrehajtja azt az operációt, amin előzőleg blokkolódott.

$\text{Up}(P)$

elengedi a szemafort

jelzést küld (Miért kell jelzést küldenie?)

Szemaforok

- Bináris szemafor- 1 biten tárolja, hogy foglalt-e (**mutex**)
- **mutex** - MUTual Exclusion
- Általános szemafor- pozitív egész szám

Szemaforok alkalmazása:

- Kölcsönös kizárás, kritikus szakasz védelme
- Szinkronizációs problémák megoldása

Mutex

A szemafor egyszerűsített változata(kétféle állapotban lehet, zárolt vagy nem zárolt)

Két művelet lehetséges:

- `mutex_lock`
- `mutex_unlock`

Ha a `mutex_lock` műveletet szeretnénk végrehajtani és a mutex nem zárolt a processzus tovább fut és beléphet a saját kritikus részébe.

A `mutex_unlock` nem blokkolósos művelet.

Kritikus szakasz védelme

```
shared semaphore s = 1;
processz-i() {
    while (true) {
        nem-kritikus-szakasz1();
        DOWN(s);
        kritikus-szakasz();
        UP(s);
        nem-kritikus-szakasz2();
    }
}
```

```
processz-i() {
    while (true) {
        nem-kritikus-szakasz1();
        DOWN(s);
        kritikus-szakasz();
        UP(s);
        nem-kritikus-szakasz2();
    }
}
```

```
#define N 100
typedef int semafor
semafor mutex=1
semafor ures=N;
semafor tele=0;
```

Termelő fogyasztó mevalósítása szemaforokkal

```
void termelo(void)
{
    int elem;
    while(TRUE)
    {
        elem=gyartas();
        down(&ures);
        down(&mutex);
        elemet_beir(elem);
        up(&mutex);
        up(&tele);
    }
}
```

```
void fogyaszto(void)
{
    int elem;
    while(TRUE)
    {
        down(&tele);
        down(&mutex);
        elem=elemet_kiolvas();
        up(&mutex);
        up(&ures);
        elemet_feldolgoz(elem);
    }
}
```

POSIX – Portable Operating System Interface System V

- [Overview](#)
- [3 Versions](#)
- [3.1 Parts before 1997](#)
- [3.2 Versions after 1997](#)
 - [3.2.1 POSIX.1-2001](#)
 - [3.2.2 POSIX.1-2004 \(with two TCs\)](#)
 - [3.2.3 POSIX.1-2008 \(with two TCs\)](#)
 - [3.2.4 POSIX.1-2017](#)

https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_system_v_posix.htm

<https://en.wikipedia.org/wiki/POSIX>

System V	POSIX
Üzenetsor	
msgget(), msgsnd(), msgrcv(), msgctl()	mq_open(), mq_send(), mq_receive(), mq_unlink()
Szemafor	
semget(), semop(), semctl()	Named Semaphores sem_open(), sem_close(), sem_unlink(), sem_post(), sem_wait(), sem_trywait(), sem_timedwait(), sem_getvalue()
	Unnamed or Memory based semaphores sem_init(), sem_post(), sem_wait(), sem_getvalue(), sem_destroy()
Osztott memória	
shmget(), shmat(), shmdt(), shmctl()	shm_open(), mmap(), shm_unlink()

POSIX Threads

pthreads C: típusok, függvények és konstansok halmaza (**pthread.h** és a thread [library](#)).

Eljárások **pthread_**

- szál menedzsment – szálak létrehozása, kapcsolódás a szálra.
- Mutex-es
- [Feltételes változók \(condition variables\)](#)
- Szálak közötti szinkronizáció (read/write locks)

POSIX [semaphore](#) API használja a POSIX threads
sem_ pthread_.

POSIX Threads

```
pthread_mutex_t lock;
```

```
pthread_mutex_init(&lock, NULL);
```

```
pthread_mutex_lock(&lock);
```

```
pthread_mutex_unlock(&lock);
```

```
pthread_mutex_destroy(&lock);
```

Általános szemafor UNIX/LINUX

```
int semget(key_t key, int nsems, int semflg);
```

```
semflag IPC_CREAT, IPC_EXLC,
```

```
int flag = IPC_CREAT | 0660;
```

```
semid = semget(key,2,IPC_CREAT|0660);
```

semop – függvénnnyel végzünk műveletet a szemaforon.

```
int semop ( int semid, struct sembuf * sops, unsigned nsops );
```

```
struct sembuf {  
    ushort sem_num; /* hanyadik szemaforra vonatkozik */  
    short sem_op;    /* a muvelet +/-n vagy 0 */  
    short sem_flg;    /* jelzo bitek: IPC_NOWAIT vagy SEM_UNDO */  
};
```

semctl

Milyen értéket vehet a sem_op

- Ha a `sem_op < 0`, akkor
vár, amíg a `semval ≥ |sem_op|`, és `semval+=sem_op`;
(Negatív `sem_op` → dekrementáció, down operáció.)
- Ha `sem_op == 0` akkor
vár, amíg `semval == 0` lesz. (Null ellenőrzés.)
- Ha `sem_op > 0`, akkor
`semval+= sem_op`;
(Pozitív `sem_op` → inkrementáció, up operáció.)

sem_flg értéke

Lehet 0, vagy vagy kapcsolatban a következő makrók

IPC_SEMUNDO

- HA a processzus befejeződik az elemi operáció „visszaáll”.

IPC_NOWAIT

- Ha nem sikerülne a down műveletet végrehajtani, akkor a semop() felfüggesztődik és a processzus fut tovább.

```
...  
while( 1)  {  
    if ( semop(..... ) )  
        { kritikus_szakasz( ); break;}  
    else  
        hasznos_feladat_elvezese( );  
}  
...
```

POSIX SEMAFOR (unnamed semaphore)

```
#include <semaphore.h>
```

```
sem_t *sem_open(const char *name, int oflag);
```

```
sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
```

Oflag **O_CREAT** –ha nem létezik létrehozza, ha létezik,

Ha **O_CREAT** és **O_EXCL**-t is meghatározzuk, abban az esetben ha egy adott nevű szemafor már létre volt hozva a *sem_open* hibát térít vissza

Ha **O_CREAT** megvan határozva, a *mode* és a *value* paramétereket is meg kell határozni

mode- hozzáférési jogok

Value –szemafor kezdeti értéke

```
sem1=sem_open("SEM_1", O_CREAT,0660,5);
```

POSIX SZEMAFAOR

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

sem -szemafor

value - szemafor kezdeti értéke

pshared - szálak vagy folyamatok között legyen megosztva a szemafor

0 -a szemafor szálak között van megosztva

Ha nem 0 -a szemafor folyamatok között van megosztva, osztott memóriában kell legyen tárolva, hogy mindkét folyamat hozzáférjen

```
status= sem_wait(&sem1);
```

```
int sem_trywait(sem_t *sem);
```

```
sem_post(&sem1);
```

Link with -pthread

```
int sem_close(sem_t *sem);
```

```
int sem_destroy(sem_t *sem);
```

```
int sem_getvalue(sem_t *sem,  
int *sval);
```

POSIX SZEMAFOR (unnamed semaphore)

```
int shm_open(const char *name, int oflag, mode_t mode);
```

```
int shm_unlink(const char *name);
```

POSIX szemafor

```
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    sem_t sem1;
    int status;
    if (argc<2)
    {printf("Hianyos argumentum");
    return -1;}

    //sem1=sem_open("SEM_1",
    O_CREAT,0660,5);
    sem_init(&sem1,1,1);
```

```
    while(1)
    {
        if ((status= sem_wait(&sem1))==-1)
        {
            printf("an error: %s\n", strerror(errno));
            exit(-1);
        }

        printf("Eslo folyamat %s\n", argv[1]);
        sleep(2);

        sem_post(&sem1);
    }
    return 0;
}
```

Monitorok

- A monitor eljárások, változók, adatszerkezetek együttese, amelyek egy speciális fajta modulba vagy csomagba vannak összegyűjtve
- A processzusok bármikor hívhatják a monitorban levő eljárásokat, de nem érhetik el a monitor belső adatszerkezeteit a monitoron kívül deklarált eljárásokból.
- Minden időpillanatban csak egy processzus lehet aktív egy monitorban
- Amikor egy processzus meghív egy monitoreljárást az eljárás első soraiban ellenőrizni fogja, hogy más processzus jelenleg **aktív-e** a Monitorban.

Monitorok (folytatás)

- Ha egy processzus eljárása jelenleg aktív egy Monitorban, a processzus felfüggesztődik, amíg a monitorban levő el nem hagyja a monitor
- A fordítóprogramtól függ, hogy van megvalósítva a kölcsönös kizárás (szemafor, mutex), de ezt már nem a programozónak kell megtervezni!
- Állapotváltozók használata: az állapotváltozón két művelet végezhető (wait, signal)
- Amikor egy monitoreljárás rájön, hogy nem léphet tovább elvégez(végrehajt) egy wait műveletet az állapotváltozón.
- A monitorból kilépő eljárás küld egy jelzést (signal)

Monitor eljárások (C++)

- (Visual C++) **MONITOR**
- (Enter) **ENTER** – megszerzi az objektumra a kizárólagos zárt
- (Exit) **EXIT** – az objektumra felszabadítja a kizárólagos
- (Pulse) **PULSE** – értesít egy szálat a várakozási sorban a zárolt objektum állapotának megváltozásáról
- (Pulse All) **PULSE ALL** – értesíti az összes várakozó szálat az objektum állapotának megváltozásáról
- (Try Enter) **TRY ENTER** – megpróbálja megszerezni a kizárólagos zárt az objektumra
- **Wait** – felszabadítja az objektum zárját, és blokkolja az aktuális szálat, amíg újra megszerzi a zárat
- **Exit** – felszabadítja az objektumra a zárat

<https://docs.microsoft.com/en-us/dotnet/api/system.threading.monitor?view=netframework-4.8>

```
// Define the lock object.  
var obj = new Object();  
// Define the critical section. Monitor.Enter(obj);  
try  
{  
    // Code to execute one thread at a time.  
  
}  
// catch blocks go here.  
finally  
{  
    Monitor.Exit(obj);  
}
```

<https://docs.microsoft.com/en-us/dotnet/api/system.threading.monitor.enter?view=netframework-4.8>

Monitor GyartoFogyaszto
condition tele, ures
int szamlalo;

```
void beszur()  
{  
    if (szamlalo==N ) wait(tele);  
    elemet_beszur(elem);  
    szamlalo=szamlalo+1;  
    if (count ==1) signal(ures);  
}
```

```
void kivesz()  
{  
    if (szamlalo == 0 wait(ures);  
    elem=elem_kivesz;  
    szamlalo=szamlalo-1;  
    if (szamlalo==N-1) signal(tele)  
}
```

```
void termelo()  
{  
    while(1)  
    {  
        elem=gyart();  
        GyartoFogyaszto.Beszur(elem);  
    }  
}
```

```
void fogyaszto()  
{  
    while(1)  
    {  
        elem=GyartoFogyaszto.Kivesz();  
        elemet_feldolgoz(elem)  
    }  
}
```

Üzenetküldés

`send(destination,&message)`

`receive(source,&message)`

Mi történik ha az üzenet nem érkezik meg? (ha a fogadó megkapta az üzenetet küld egy válaszüzenetet(nyugtázza a küldővel)

De mi történik abban az esetben ha elvész a nyugta?

pl. minden üzenethez hozzárendelünk egy azonosítót és ha az előbbi elküldött üzenetre nem érkezett nyugta (egy meghatározott idő alatt), elküldjük másodszorra az üzenetet, a fogadó az azonosító alapján eldönti, hogy azt az üzenetet már az előbbi lépésben megkapta.

Gyártó fogyasztó probléma üzenetküldéssel

```
void gyarto()
{
int elem;
uzenet m;
while(TRUE)
{   elem=gyartas();
    receive(fogyaszto,&m);
    uzenet_felepit(&m,elem);
    send(fogyaszto,&m)
}
}
```

```
void fogyaszto()
{
int elem; uzenet m;
for(i=0;i<N;i++)
    send(gyarto,&m)
while(TRUE)
{   receive(gyarto,&m);
    elem=elemet_kivesz();
    send(gyarto,&m)
    elemet_feldolgoz(elem);
}
}
```

- **msg.h**
- `msgid = msgget((key_t) KEY, IPC_CREAT | 0666);`
- `msgid = msgget((key_t) KEY, 0);`
- `int msgctl(int msgid, int cmd, struct msqid_ds *buf)`
- `IPC_STAT, IPC_SET, IPC_RMID`
- `int msgsnd(int msgid, const void *addr, size_t nbytes, int flg);`
- `int msgrcv(int msgid, void *addr, size_t nbytes, long msgtype, int flg);`
 - `msgtype = 0` – a sor első üzenetét olvassa ki,
 - `msgtype > 0` – a sor első `msgtype` típusú üzenetét olvassa ki,
 - `msgtype < 0` – a sor legkisebb, de az `abs(msgtype)`-nál nagyobb vagy azzal egyenlő értékű üzenetét olvassa ki.

```
struct msg
{
    long tip;
    char uzenet[256];
};
```

Osztott memória

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```
int shmid;
shmid = shmget((key_t) KEY, 200, IPC_CREAT | 0666);
shmid = shmget((key_t) KEY, 0, 0); -nem hoz létre új sort, ha egy már korábban létrehozott
osztott memória ID-ját szeretnénk meghatározni
```

```
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

- IPC_STAT, IPC_SET, IPC_RMID
- SHM_LOCK, SHM_UNLOCK

```
shmctl(shmid, IPC_RMID, NULL); -az osztott memória törlése
```

```
void *shmat(int shmid, void *addr, int flg);
```

```
int shmdt(void *addr);
```